

Flood Prediction System

Rising waters

Project Description:

Rising Waters is a web-based Machine Learning application developed to predict the likelihood of floods using historical weather and rainfall data. The system assists users in estimating flood risk by analyzing important environmental parameters such as annual rainfall, seasonal rainfall, temperature, humidity, cloud cover, and average rainfall.

The project uses several Machine Learning algorithms including Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and XGBoost. After training and evaluating all models, the best-performing model is selected and integrated into a Flask web application. The trained model is saved using Joblib and loaded during prediction to provide fast and accurate results. The application is built using Python, Flask, HTML, CSS, JavaScript, Scikit-learn, Pandas, NumPy, and XGBoost. It provides a simple and interactive user interface where users enter weather-related values and instantly receive the flood prediction result. The project has been deployed online using Render, making it accessible from any device with an internet connection.

Problem Statement

Floods are among the most destructive natural disasters, causing significant damage to lives, agriculture, infrastructure, and the environment. Traditional flood warning systems often require extensive monitoring equipment and may not always provide quick predictions for changing weather conditions.

The objective of this project is to develop a Machine Learning-based Flood Prediction System that analyzes weather and rainfall parameters to predict the likelihood of flooding. The system provides quick predictions through a user-friendly web application, helping users understand potential flood risks in advance.

Project Objectives

The main objectives of this project are:

- Develop a Machine Learning model capable of predicting flood occurrence.
- Analyze multiple weather and rainfall parameters.
- Provide real-time flood prediction through a web application.
- Build an easy-to-use interface for users.
- Deploy the application on the cloud for public accessibility.

Project Scope

The Flood Prediction System is designed to assist users in obtaining an early indication of flood risk based on weather conditions.

The project covers:

- Weather data analysis
- Flood prediction
- Machine Learning model deployment
- Web-based prediction interface
- Cloud deployment using Render

The current system predicts whether flood conditions are likely but does not provide geographical mapping or real-time weather integration.

Scenarios

Scenario 1 – Farmer

A farmer enters the current weather information before the monsoon season. The application predicts a high probability of flooding, allowing the farmer to take preventive measures to protect crops.

Scenario 2 – Disaster Management Team

Emergency management officials analyze rainfall conditions using the application. A flood prediction enables them to prepare rescue teams and issue early warnings.

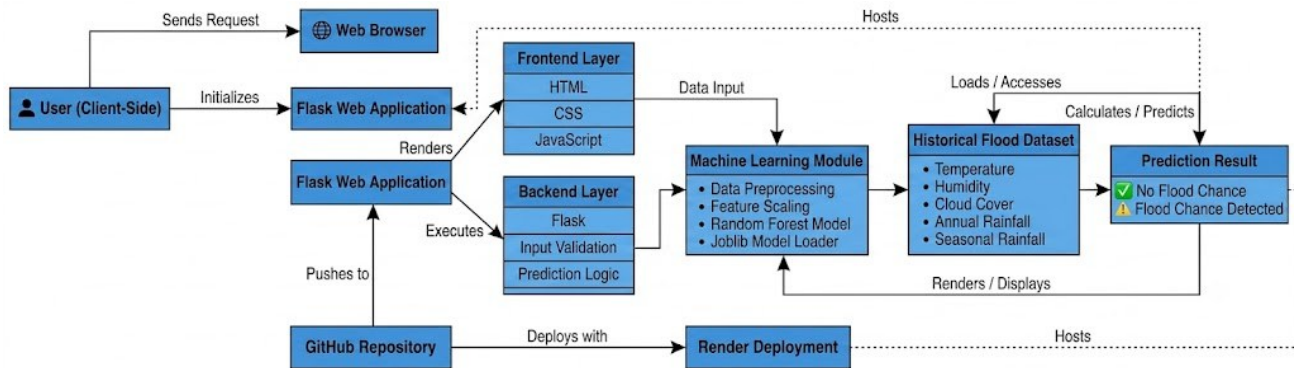
Scenario 3 – Research Student

A student studying environmental science uses different rainfall values to understand how weather parameters influence flood prediction.

Scenario 4 – Public Awareness

A citizen checks weather conditions through the application before traveling to a flood-prone area and receives a flood risk prediction.

Technical Architecture:



Technical Architecture Description

The technical architecture of the **Flood Prediction System** illustrates the overall workflow of the application, showing how different components interact to generate flood predictions. The system is developed using a Flask-based web application integrated with an XGBoost Machine Learning model.

The process begins with the **Flood Prediction System**, which serves as the central application. The system is built using the **Flask Web Application Framework**, providing the backend functionality and handling communication between the frontend and the Machine Learning model.

Users interact with the system by entering weather and rainfall parameters through the prediction interface. These **user inputs** are received by the application and forwarded to the trained **XGBoost Machine Learning model** after appropriate preprocessing and scaling using the saved StandardScaler. The application also loads the previously trained model and scaler from the **Joblib files**, ensuring that the same preprocessing steps used during training are applied during prediction.

The XGBoost model analyzes the processed input data and generates prediction results. Based on the model's output, the system provides three types of information to the user:

- **Flood Prediction Result**, indicating whether flooding is likely or not.
- **Risk Level**, categorizing the situation as either *Safe* or *High Risk*.
- **Recommendations**, providing suitable guidance based on the predicted flood risk.

This architecture demonstrates the integration of the Flask backend, Machine Learning model,

Pre-requisites:

- Python Programming
- Machine Learning Fundamentals
- Flask Framework
- HTML
- CSS
- JavaScript
- Pandas
- NumPy
- Scikit-learn
- XGBoost
- Joblib
- Git & GitHub
- Render Deployment

Project Workflow:

Project Milestones

The development of the Flood Prediction System was completed in five major milestones. Each milestone focused on a specific phase of the Machine Learning and Web Application development process.

Milestone 1: Project Planning & Environment Setup

The first milestone focused on establishing a strong foundation for the Flood Prediction System. During this phase, the project requirements were analyzed, and the problem statement was clearly defined. A suitable historical flood dataset containing weather and rainfall parameters was collected for machine learning model development. After understanding the project objectives, the technology stack was finalized, including Python for programming, Flask for web development, XGBoost for machine learning, HTML and CSS for the user interface, and Render for cloud deployment. The development environment was then configured by installing the required software and libraries, creating a virtual environment, and organizing the project into a modular folder structure. Successfully completing this milestone provided a well-organized environment for the remaining stages of development.

Milestone 2: Data Preprocessing & Model Development

The second milestone concentrated on preparing the dataset and developing the machine learning model. The collected dataset was thoroughly analyzed to understand its structure and quality. Data preprocessing techniques such as checking for missing values, removing duplicates, selecting relevant features, and applying feature scaling were performed to improve the quality of the input data. Multiple machine learning algorithms, including Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and XGBoost, were trained and evaluated using the prepared dataset. Their performances were compared using evaluation metrics, and XGBoost was selected as the final model because it achieved the highest prediction accuracy. The trained model and StandardScaler were then saved using Joblib for integration into the Flask application.

Milestone 3: Flask Web Application Development

The third milestone focused on converting the trained machine learning model into a user-friendly web application. A responsive frontend was developed using HTML and CSS, allowing users to enter weather and rainfall information through an intuitive prediction form. The backend was built using Flask, which receives the user inputs, validates the entered values, preprocesses the data using the saved StandardScaler, and sends it to the trained XGBoost model for prediction. The prediction result, indicating whether flooding is likely or not, is then displayed instantly on the same webpage. This milestone successfully integrated machine learning with web technologies to create an interactive prediction system.

Milestone 4: Testing & Performance Evaluation

The fourth milestone involved testing the complete application to verify its functionality, performance, and reliability. Various combinations of valid weather parameters were entered into the system to ensure that the application generated accurate predictions. Input validation was tested to prevent invalid or out-of-range values from being submitted. The application's response time, prediction accuracy, and overall performance were evaluated under normal operating conditions. Minor issues identified during testing were corrected, and the user interface was refined to improve usability. The successful completion of this milestone ensured that the application was stable and ready for deployment.

Milestone 5: Deployment & Documentation

The final milestone focused on deploying the Flood Prediction System and completing all project documentation. The Flask application was deployed on the Render cloud platform, making it accessible through a web browser without requiring local installation. A GitHub repository was created to maintain the complete project source code, documentation, and version history. A detailed README file was prepared, including project information, installation steps, technologies used, project structure, and deployment details. Finally, testing reports, screenshots, and all required project documentation were compiled. This milestone marked the successful completion of the project and ensured that it was ready for demonstration, evaluation, and future enhancements.

Milestone 1: Project Planning & Environment Setup

1.1 Introduction

The first milestone focused on understanding the project requirements and preparing the development environment. Before building the Machine Learning model, it was necessary to identify the problem, define clear objectives, collect an appropriate dataset, and select the technologies required for development. A proper development environment was then configured to ensure that all required libraries and tools were installed correctly. This milestone established the foundation for the successful implementation of the Flood Prediction System.

1.2 Problem Identification

Floods are one of the most common natural disasters and often result in significant damage to lives, property, agriculture, and infrastructure. Although weather forecasts provide rainfall information, they do not always indicate whether flood conditions are likely to occur.

To address this problem, a Machine Learning-based Flood Prediction System was proposed. The objective was to develop a web application capable of analyzing historical weather parameters and predicting whether flooding is likely.

1.3 Project Objectives

The objectives identified during this milestone are:

- Develop a Machine Learning model for flood prediction.
- Analyze historical rainfall and weather data.
- Compare multiple Machine Learning algorithms.
- Select the best-performing model.
- Develop an interactive Flask web application.
- Deploy the application on Render.
- Provide accurate flood prediction in real time.

1.4 Dataset Collection

The next step involved collecting the dataset required for model development.

The dataset contains historical weather information including temperature, humidity, cloud cover, annual rainfall, seasonal rainfall, average June rainfall, subdivision rainfall, and the corresponding flood occurrence label.

The dataset was stored in Excel format and later imported into Python using the Pandas library.

1.5 Technology Stack Selection

The following technologies were selected for implementing the project.

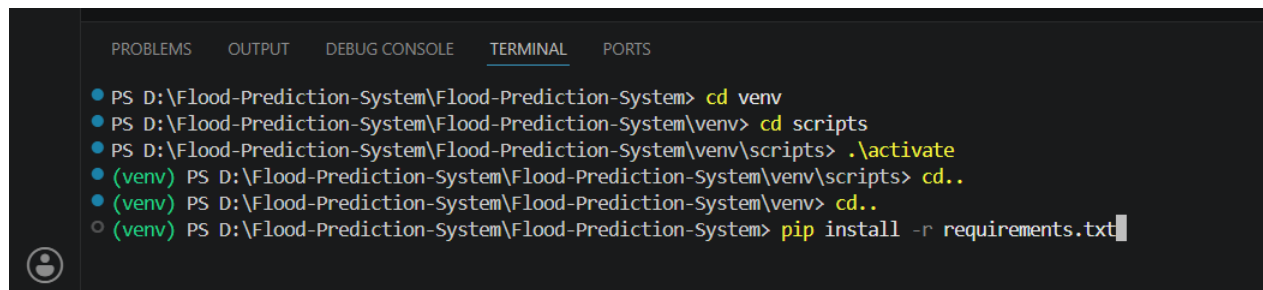
Technology	Purpose
Python	Programming
Pandas	Dataset Processing
NumPy	Numerical Computation
Matplotlib	Visualization
Seaborn	Data Analysis
Scikit-learn	Preprocessing
XGBoost	Machine Learning
Flask	Backend
HTML	Frontend
CSS	Styling
GitHub	Version Control
Render	Deployment

1.6 Development Environment Setup

The project was developed using Visual Studio Code.

The following setup was completed.

- Python installed
- VS Code installed
- Virtual Environment created
- Required libraries installed
- Git initialized
- GitHub repository created



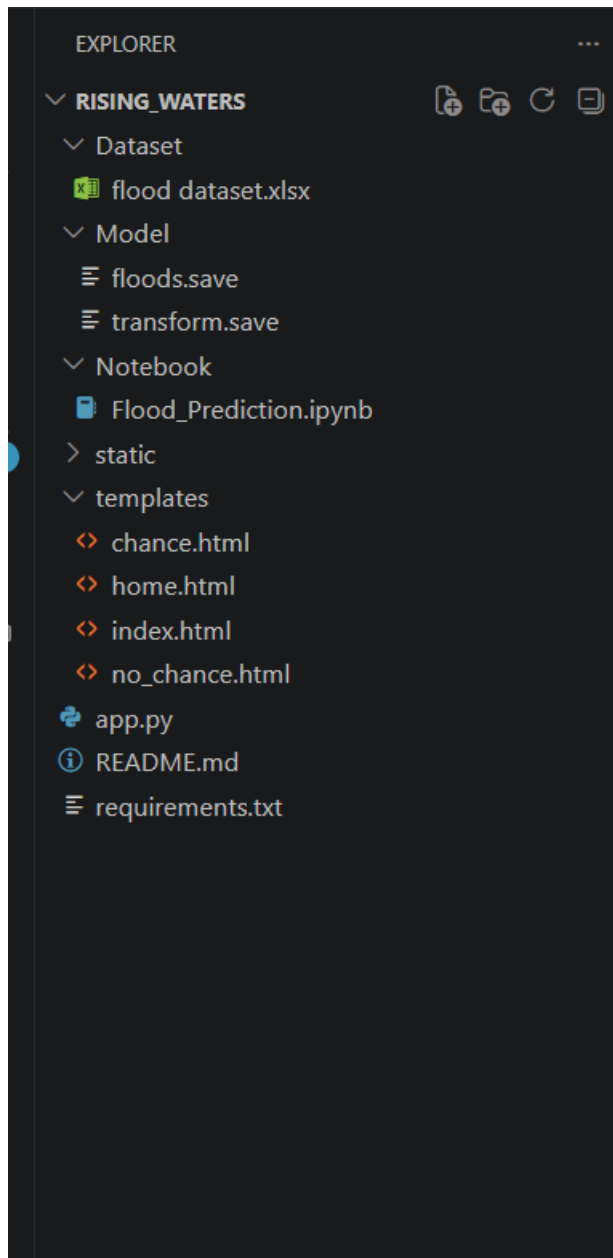
```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
• PS D:\Flood-Prediction-System\Flood-Prediction-System> cd venv
• PS D:\Flood-Prediction-System\Flood-Prediction-System\venv> cd scripts
• PS D:\Flood-Prediction-System\Flood-Prediction-System\venv\scripts> .\activate
• (venv) PS D:\Flood-Prediction-System\Flood-Prediction-System\venv\scripts> cd..
• (venv) PS D:\Flood-Prediction-System\Flood-Prediction-System\venv> cd..
• (venv) PS D:\Flood-Prediction-System\Flood-Prediction-System> pip install -r requirements.txt
  
```

1.7 Project Folder Structure

After configuring the environment, the project was organized into multiple folders.

Flood-Prediction-System/



1.8 Render Account Setup

A Render account was created for deploying the Flask application.

The GitHub repository was connected with Render to enable automatic deployment.

1.9 Deliverables

The following tasks were completed successfully.

- Problem Statement finalized
- Dataset collected
- Technologies selected
- Environment configured
- GitHub repository created
- Render account configured
- Folder structure organized
- Libraries installed

1.10 Challenges Faced

During the setup process, some Python packages required compatible versions to avoid dependency conflicts. A virtual environment was therefore created to isolate project dependencies. Organizing the project into separate folders also required careful planning to ensure future maintainability.

1.12 Outcome

At the end of Milestone 1, the complete development environment was ready. The dataset had been collected, the technology stack finalized, the project structure created, and all required tools installed successfully. This milestone provided the foundation for developing and training the Machine Learning model in the next phase.

Milestone 2 : Data Preprocessing & Machine Learning Model Development

2.1 Introduction

After successfully setting up the project environment, the next step was to prepare the dataset and develop a Machine Learning model capable of predicting flood occurrence. This milestone involved understanding the dataset, preprocessing the data, performing exploratory data analysis, training multiple Machine Learning algorithms, evaluating their performance, and selecting the most suitable model for deployment.

Proper preprocessing plays a significant role in improving the prediction accuracy of any Machine Learning model. Therefore, considerable attention was given to data cleaning, feature scaling, model evaluation, and performance comparison.

2.2 Importing Required Libraries

The first step involved importing all the necessary Python libraries required for data manipulation, visualization, preprocessing, model training, evaluation, and model saving.

The following libraries were imported:

- Pandas
- NumPy
- Matplotlib
- Seaborn
- Scikit-learn
- XGBoost
- Joblib

These libraries provide various functionalities required throughout the Machine Learning workflow.

```
# =====  
# Import Required Libraries  
# =====  
  
import numpy as np          # Numerical calculations  
import pandas as pd         # Reading and handling datasets  
  
import matplotlib.pyplot as plt # Graphs  
import seaborn as sns        # Statistical graphs  
  
%matplotlib inline
```

2.3 Loading the Dataset

The dataset was stored in Excel format and loaded using the Pandas library. Once loaded, the dataset was examined to verify that it had been imported correctly.

The first few records of the dataset were displayed to understand the structure and available features.

```
dataset = pd.read_excel("../dataset/flood_dataset.xlsx")

dataset.head()
```

	Temp	Humidity	Cloud Cover	ANNUAL	Jan-Feb	Mar-May	Jun-Sep	Oct-Dec	avgjune	sub	flood
0	29	70	30	3248.6	73.4	386.2	2122.8	666.1	274.866667	649.9	0
1	28	75	40	3326.6	9.3	275.7	2403.4	638.2	130.300000	256.4	1
2	28	75	42	3271.2	21.7	336.3	2343.0	570.1	186.200000	308.9	0
3	29	71	44	3129.7	26.7	339.4	2398.2	365.3	366.066667	862.5	0
4	31	74	40	2741.6	23.4	378.5	1881.5	458.1	283.400000	586.9	0

2.4 Understanding the Dataset

Understanding the dataset before training the model is essential. Information about the dataset such as the number of records, column names, data types, and summary statistics was analyzed. The following analyses were performed:

- Dataset dimensions
- Data types
- Statistical summary
- Feature names

This helped in identifying the characteristics of each feature before preprocessing.

```
dataset.info()
```

```
[7]
... <class 'pandas.DataFrame'>
RangeIndex: 115 entries, 0 to 114
Data columns (total 11 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   Temp            115 non-null   int64
1   Humidity        115 non-null   int64
2   Cloud Cover     115 non-null   int64
3   ANNUAL          115 non-null   float64
4   Jan-Feb         115 non-null   float64
5   Mar-May         115 non-null   float64
6   Jun-Sep         115 non-null   float64
7   Oct-Dec         115 non-null   float64
8   avgjune         115 non-null   float64
9   sub             115 non-null   float64
10  flood           115 non-null   int64
dtypes: float64(7), int64(4)
memory usage: 10.0 KB
```

```
dataset.describe()
```

Python

	Temp	Humidity	Cloud Cover	ANNUAL	Jan-Feb	Mar-May	Jun-Sep	Oct-Dec	avgjune	sub	flood
count	115.000000	115.000000	115.000000	115.000000	115.000000	115.000000	115.000000	115.000000	115.000000	115.000000	115.000000
mean	29.600000	73.852174	36.286957	2925.487826	27.739130	377.253913	2022.840870	497.636522	218.100870	439.801739	0.139130
std	1.122341	2.947623	4.330158	422.112193	22.361032	151.091850	386.254397	129.860643	62.547597	210.438813	0.347597
min	28.000000	70.000000	30.000000	2068.800000	0.300000	89.900000	1104.300000	166.600000	65.600000	34.200000	0.000000
25%	29.000000	71.000000	32.500000	2627.900000	10.250000	276.750000	1768.850000	407.450000	179.666667	295.000000	0.000000
50%	30.000000	74.000000	36.000000	2937.500000	20.500000	342.000000	1948.700000	501.500000	211.033333	430.600000	0.000000
75%	31.000000	76.000000	40.000000	3164.100000	41.600000	442.300000	2242.900000	584.550000	263.833333	577.650000	0.000000
max	31.000000	79.000000	44.000000	4257.800000	98.100000	915.200000	3451.300000	823.300000	366.066667	982.700000	1.000000

2.5 Data Cleaning

The dataset was inspected to ensure that it was suitable for model training. Missing values and duplicate records were checked, and the data types of all columns were verified.

Fortunately, the dataset was already clean, so no major cleaning operations were required.

The following validation checks were performed:

- Missing values
- Duplicate rows
- Invalid data entries

```
dataset.isnull().sum()
```

[14]

```
... Temp      0
    Humidity   0
    Cloud Cover 0
    ANNUAL     0
    Jan-Feb    0
    Mar-May    0
    Jun-Sep    0
    Oct-Dec    0
    avgjune    0
    sub        0
    flood      0
    dtype: int64
```

```
dataset.isnull().sum()
```

[14]

```
... Temp      0
    Humidity   0
    Cloud Cover 0
    ANNUAL     0
    Jan-Feb    0
    Mar-May    0
    Jun-Sep    0
    Oct-Dec    0
    avgjune    0
    sub        0
    flood      0
    dtype: int64
```

2.6 Exploratory Data Analysis (EDA)

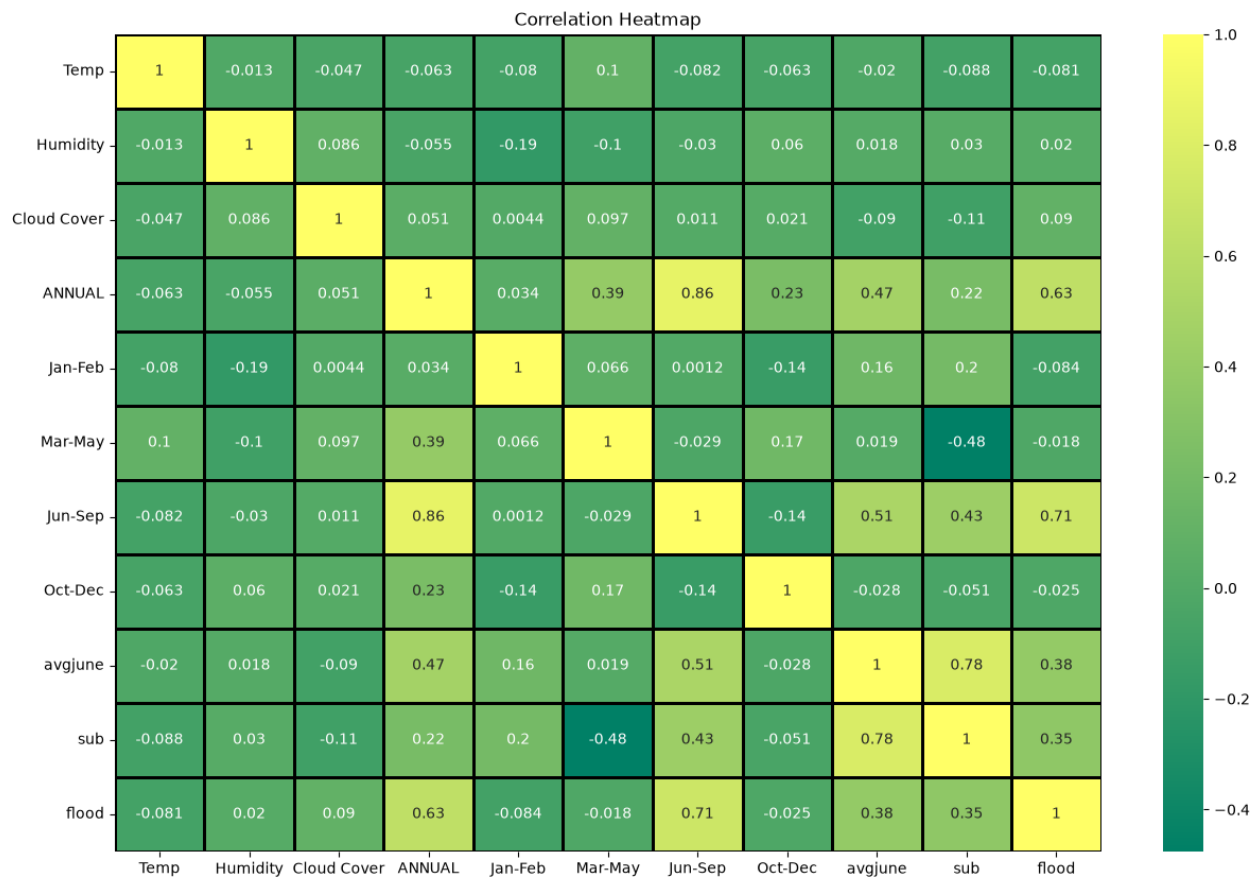
Exploratory Data Analysis was performed to understand the relationship between different weather parameters and flood occurrence.

Visualization techniques helped in identifying trends and distributions within the dataset.

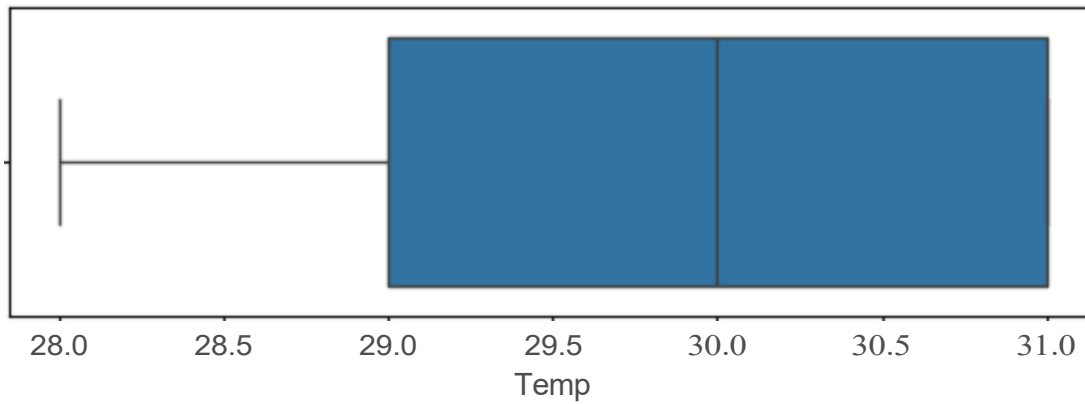
The following visualizations were created:

- Histogram
- Correlation Heatmap
- Box Plot
- Distribution Plot

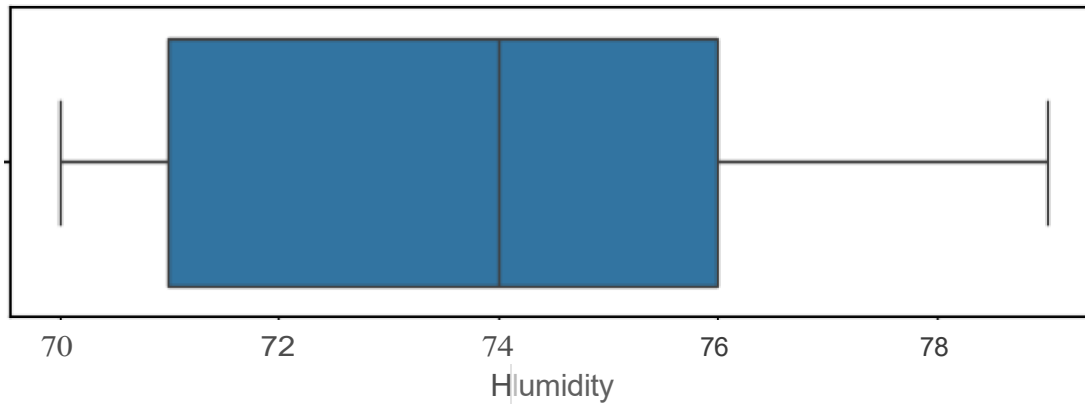
EDA improved the understanding of the dataset and supported feature selection.



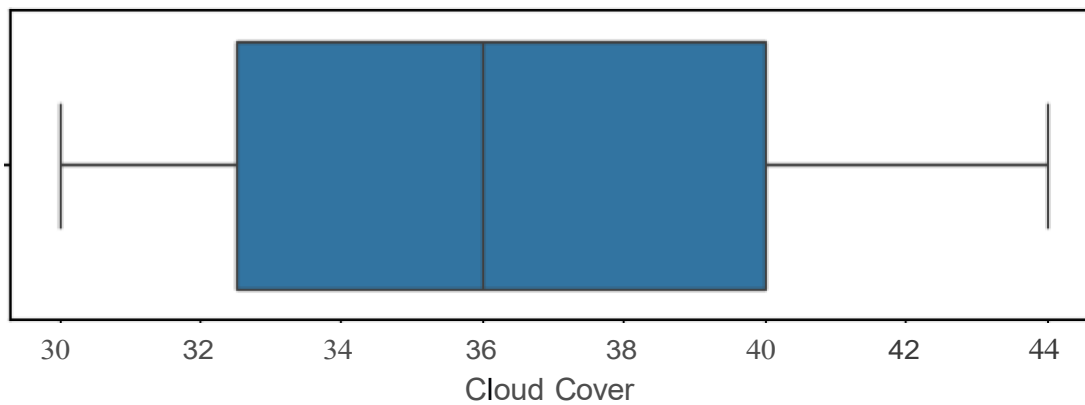
Box Plot After Capping - Temp

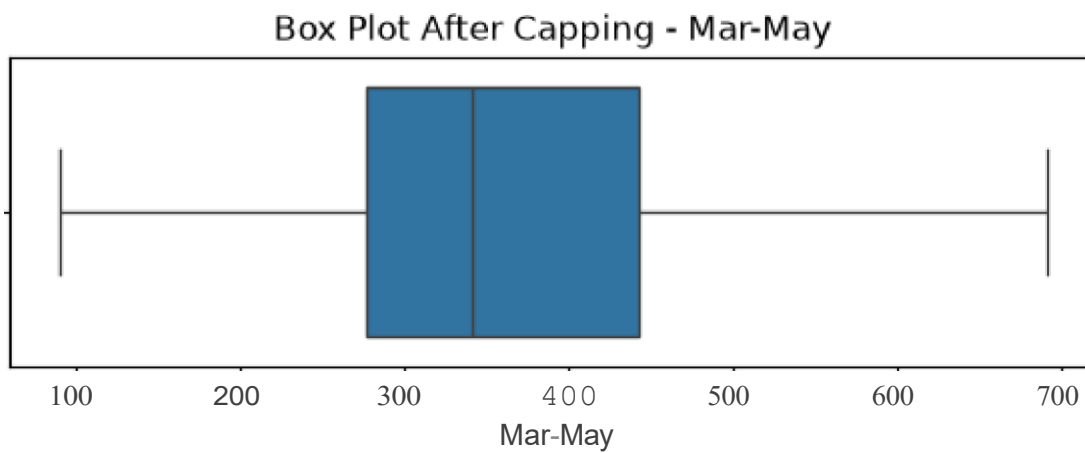
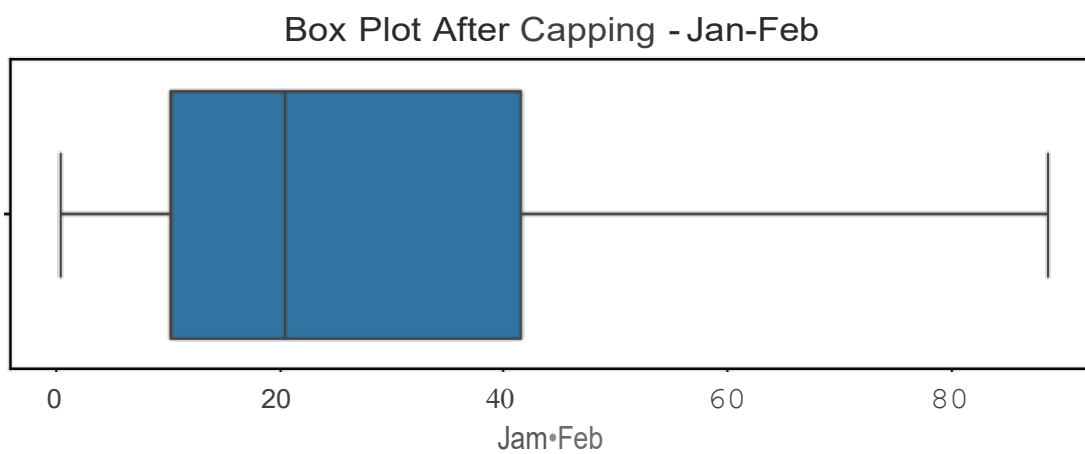
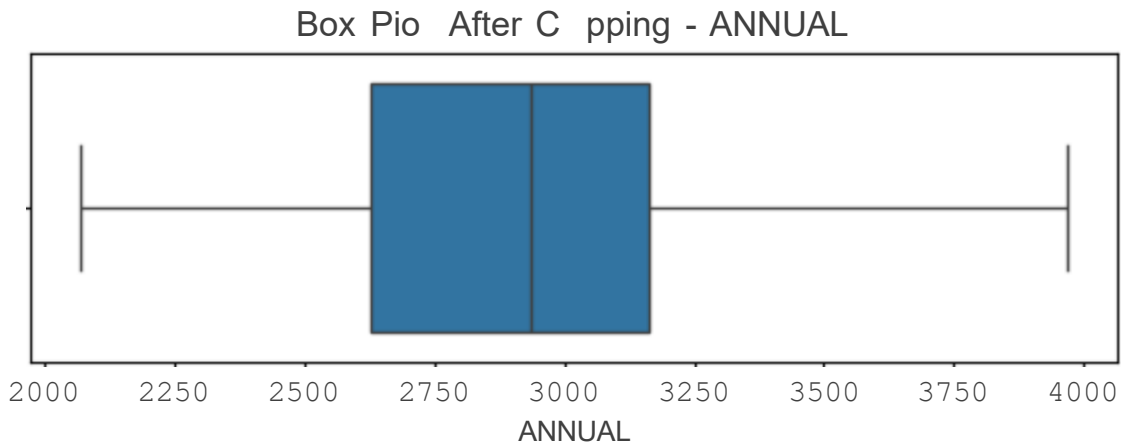


Box Plot After Capping - Humidity

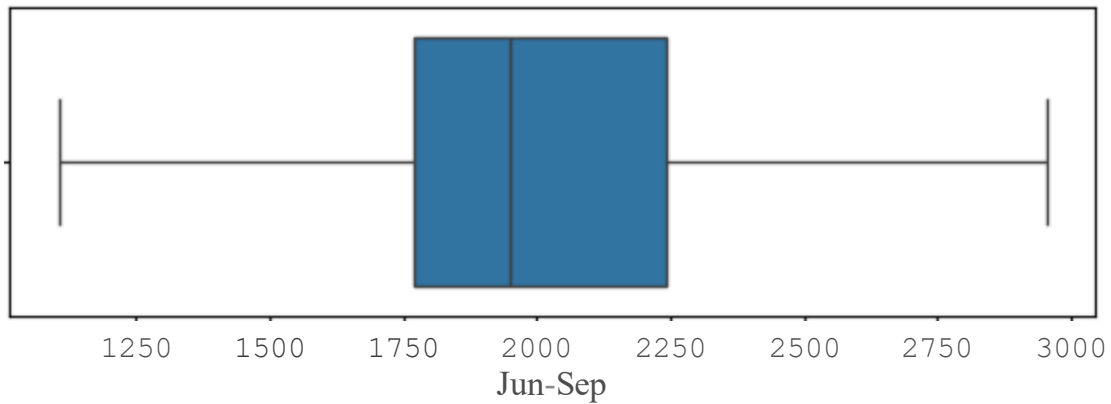


Box Plot After Capping - Cloud Cover

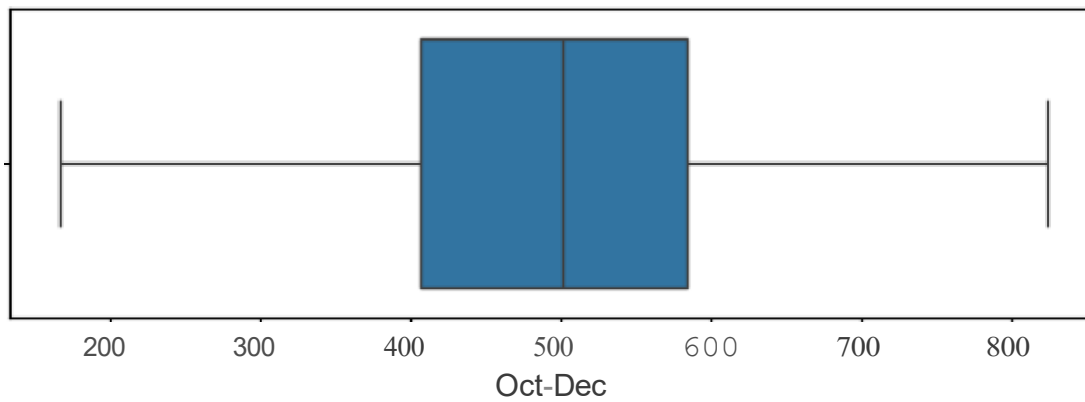




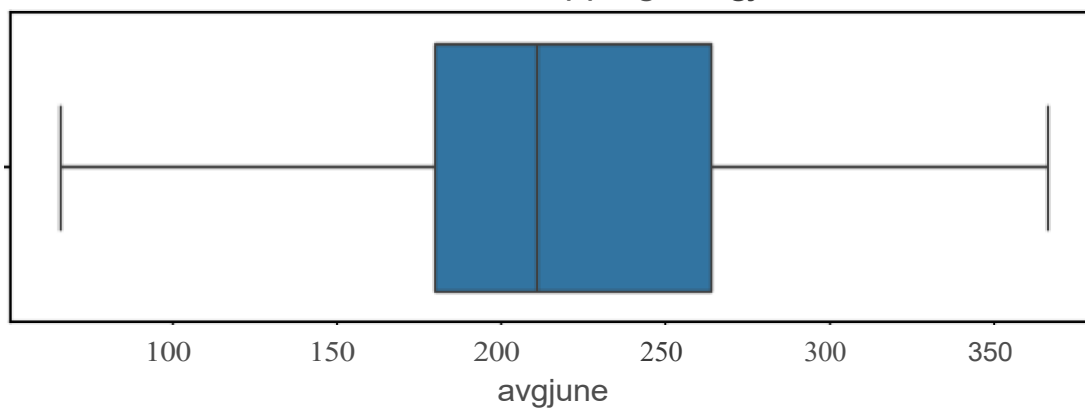
Box Plot After Capping - Jun-Sep



Box Plot After Capping - Oct-Dec



Box Plot After Capping - avgjune



2.7 Feature Selection

The input features and target variable were selected for model training.

The selected input features were:

- Temperature
- Humidity
- Cloud Cover
- Annual Rainfall
- Jan-Feb Rainfall
- Mar-May Rainfall
- Jun-Sep Rainfall
- Oct-Dec Rainfall
- Average June Rainfall
- Subdivision Rainfall

The target variable represented flood occurrence.

2.8 Train-Test Split

The dataset was divided into training and testing datasets.

A larger portion of the dataset was used for training while the remaining data was reserved for testing.

This ensures that the model is evaluated on unseen data.

```
from sklearn.model_selection import train_test_split

x_train, x_test, y_train, y_test = train_test_split(
    x,
    y,
    test_size=0.25,
    random_state=10
)
```

```
print("x_train :", x_train.shape)
print("x_test  :", x_test.shape)

print("y_train :", y_train.shape)
print("y_test  :", y_test.shape)
```

```
x_train : (86, 10)
x_test  : (29, 10)
y_train : (86,)
y_test  : (29,)
```

2.9 Feature Scaling

The dataset contains numerical values with different ranges.

To ensure uniformity, StandardScaler was applied.

Feature scaling improves the learning capability of many Machine Learning algorithms and ensures consistent preprocessing during prediction.

The fitted scaler was saved using Joblib.

```
# =====  
# Creating StandardScaler Object  
# =====  
  
sc = StandardScaler()  
  
# =====  
# Scaling Training Data  
# =====  
  
x_train = sc.fit_transform(x_train)  
  
# =====  
# Scaling Testing Data  
# =====  
  
x_test = sc.transform(x_test)
```

2.10 Deliverables

At the end of this phase,

- Dataset successfully loaded
- Dataset analyzed
- Missing values checked
- Duplicate records checked
- Exploratory Data Analysis completed
- Features selected
- Dataset split into train and test
- StandardScaler applied
- Scaler saved

Milestone 3 : Flask Web Application Development

3.1 Introduction

After successfully developing and saving the Machine Learning model in the previous milestone, the next objective was to build a web application that allows users to interact with the model. Flask was selected as the backend framework because it is lightweight, easy to integrate with Machine Learning models, and suitable for developing web applications.

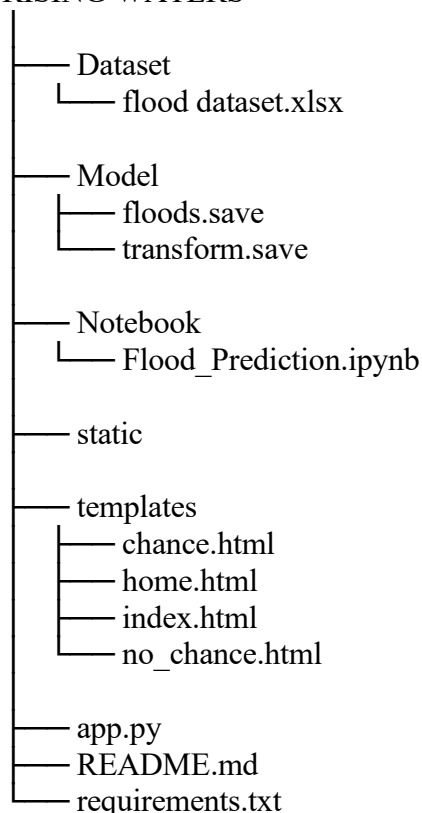
During this milestone, the trained XGBoost model and StandardScaler were integrated into the Flask application. A responsive user interface was designed using HTML and CSS, enabling users to enter weather parameters and receive flood prediction results instantly. The application was structured using Flask templates and static files to ensure maintainability and scalability.

3.2 Creating the Flask Project Structure

The first step in developing the web application was organizing the project into a modular folder structure. Separate folders were created for HTML templates, CSS files, images, datasets, trained models, and backend code. This organization makes the project easier to understand, maintain, and extend.

Project Folder Structure

RISING WATERS



3.3 Creating the Flask Application

The Flask application was initialized in `app.py`. This file acts as the main entry point of the project and is responsible for creating the Flask application instance, loading the trained Machine Learning model, defining application routes, processing user requests, and displaying prediction results.

```
app.py ×  
app.py > prediction  
1 from flask import Flask, render_template, request  
2 import joblib  
3 import numpy as np  
4  
5 app = Flask(__name__)  
6
```

3.4 Loading the Trained Model

The trained XGBoost model and the StandardScaler were loaded using the Joblib library when the Flask application starts. Loading the model only once during application startup improves prediction speed because the model does not need to be reloaded for every prediction request.

```
model = joblib.load("saved_models/flood_model.joblib")  
scaler = joblib.load("saved_models/scaler.joblib")
```

3.5 Home Route Development

The home route was created to display the landing page of the Flood Prediction System. This page introduces the application, provides information about its purpose, and allows users to navigate to the prediction page.

The home page was developed using HTML and styled using CSS to provide a clean and responsive user interface.

```
@app.route("/")  
def home():  
    return render_template("index.html")
```

New Tab

Sample Project Document.docx

Flood Prediction System

+

Ask Gemini

—

X

←

→

↻

rising-waters-19hr.onrender.com

🔍

☆

📄

🔴

⋮



Rising Waters

Machine Learning Flood Prediction System

Predict Flood Risk Before It Happens

Analyze rainfall, humidity, cloud cover and seasonal weather conditions using an intelligent Machine Learning model. Receive an instant prediction that helps identify potential flood situations.

Flood Prediction


Temperature (°C)


Humidity (%)

Example: 30

Example: 75


Cloud Cover (%)


Annual Rainfall (mm)

Example: 40

Example: 3200


Jan-Feb Rainfall (mm)


Mar-May Rainfall (mm)

Example: 250

Example: 650


Jun-Sep Rainfall (mm)


Oct-Dec Rainfall (mm)

Example: 3100

Example: 250


Average June Rainfall (mm)


Subdivision Rainfall (mm)

Example: 275

Example: 650

Predict Flood



30°C

Mostly cloudy



Q

Search

















ENG

IN

🔊

🔋

12:16 PM

04-07-2026

3.6 Prediction Page Development

A dedicated prediction page was created to collect weather and rainfall information from users. The page consists of ten input fields representing different environmental parameters required by the Machine Learning model.

The prediction form was designed using HTML, while CSS was used to provide a professional and responsive appearance.

```
<form action="/prediction" method="POST">
  <div class="form-grid">
    <div class="input-box">
      <label>🌡 Temperature (°C)</label>

      <input type="number"
        step="0.1"
        min="0"
        max="60"
        name="temp"
        placeholder="Example: 30"
        required>
    </div>

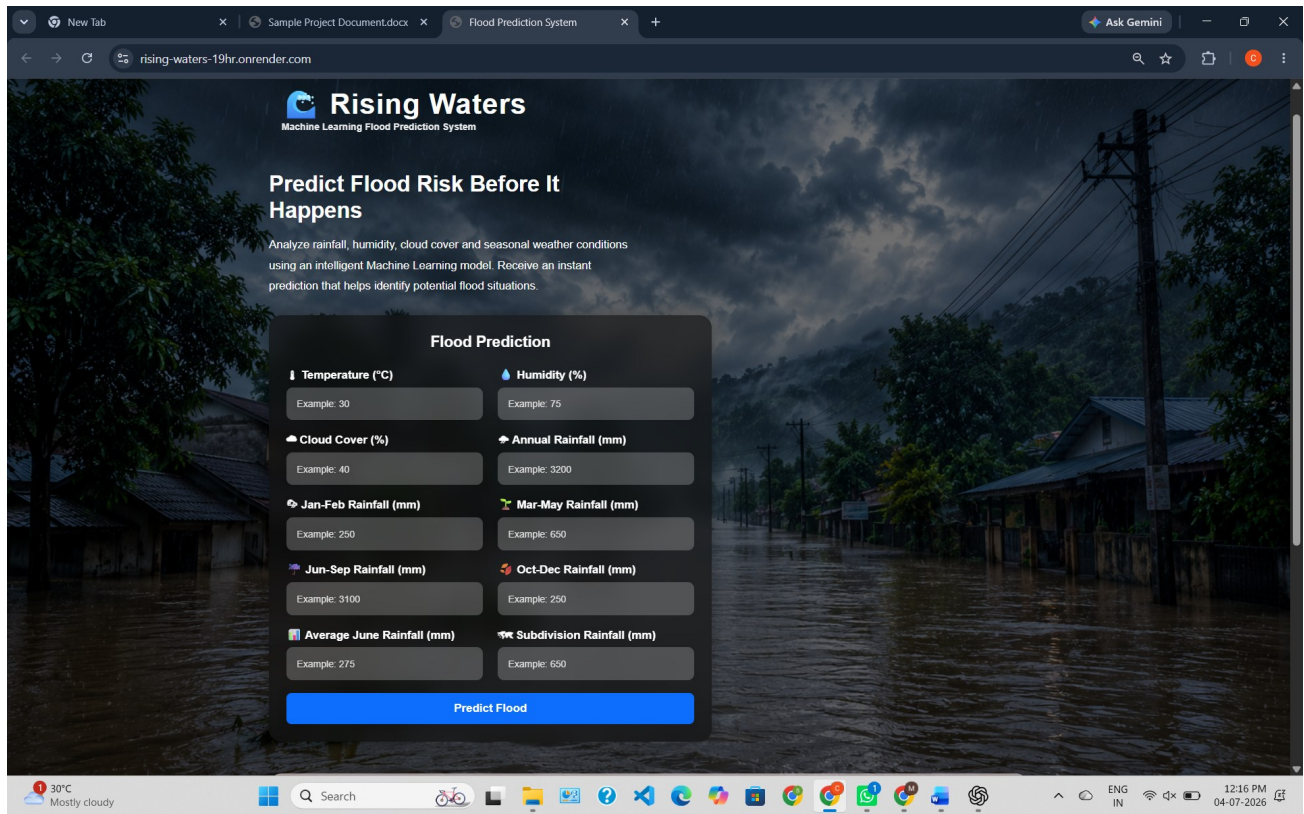
    <div class="input-box">
      <label>💧 Humidity (%)</label>

      <input type="number"
        step="0.1"
        min="0"
        max="100"
        name="humidity"
        placeholder="Example: 75"
        required>
    </div>
  </div>
</form>
```

3.7 Input Validation

To prevent invalid data from being submitted, HTML input validation was implemented. Each input field was assigned appropriate minimum and maximum values along with the required attribute.

This ensures that users can only enter realistic weather values before prediction.



Rising Waters
Machine Learning Flood Prediction System

Predict Flood Risk Before It Happens

Analyze rainfall, humidity, cloud cover and seasonal weather conditions using an intelligent Machine Learning model. Receive an instant prediction that helps identify potential flood situations.

Flood Prediction

Temperature (°C) Example: 30	Humidity (%) Example: 75
Cloud Cover (%) Example: 40	Annual Rainfall (mm) Example: 3200
Jan-Feb Rainfall (mm) Example: 250	Mar-May Rainfall (mm) Example: 650
Jun-Sep Rainfall (mm) Example: 3100	Oct-Dec Rainfall (mm) Example: 250
Average June Rainfall (mm) Example: 275	Subdivision Rainfall (mm) Example: 650

Predict Flood

3.8 Prediction Route

The prediction route receives user input from the prediction form using an HTTP POST request. The entered values are converted into numerical format, arranged into an input array, scaled using the previously saved StandardScaler, and passed to the trained XGBoost model. The predicted output is then processed and displayed on the same webpage.

```
@app.route("/prediction", methods=["POST"])
def prediction():
    try:
        # =====
        # Read Form Values
        # =====

        temp = float(request.form["temp"])
        humidity = float(request.form["humidity"])
        cloud = float(request.form["cloud"])
        annual = float(request.form["annual"])
        janfeb = float(request.form["janfeb"])
        marmay = float(request.form["marmay"])
        junsep = float(request.form["junsep"])
        octdec = float(request.form["octdec"])
        avgjune = float(request.form["avgjune"])
        sub = float(request.form["sub"])

        # =====
        # Create Input Array
        # =====

        input_data = np.array([
            temp,
            humidity,
            cloud,
            annual,
            janfeb,
            marmay,
            junsep,
            octdec,
            avgjune,
            sub
        ])
    except:
```

3.9 Data Scaling During Prediction

Before prediction, the user input is transformed using the StandardScaler that was saved during model development. This ensures that the input data has the same scale as the data used to train the model.

```
scaled_data = scaler.transform(input_data)
```

3.10 Prediction Generation

The scaled input data is passed to the trained XGBoost model to generate a prediction. The prediction result is then converted into a human-readable format.

If the predicted value is 1, the application displays **Flood Likely**. Otherwise, it displays **No Flood Likely**.

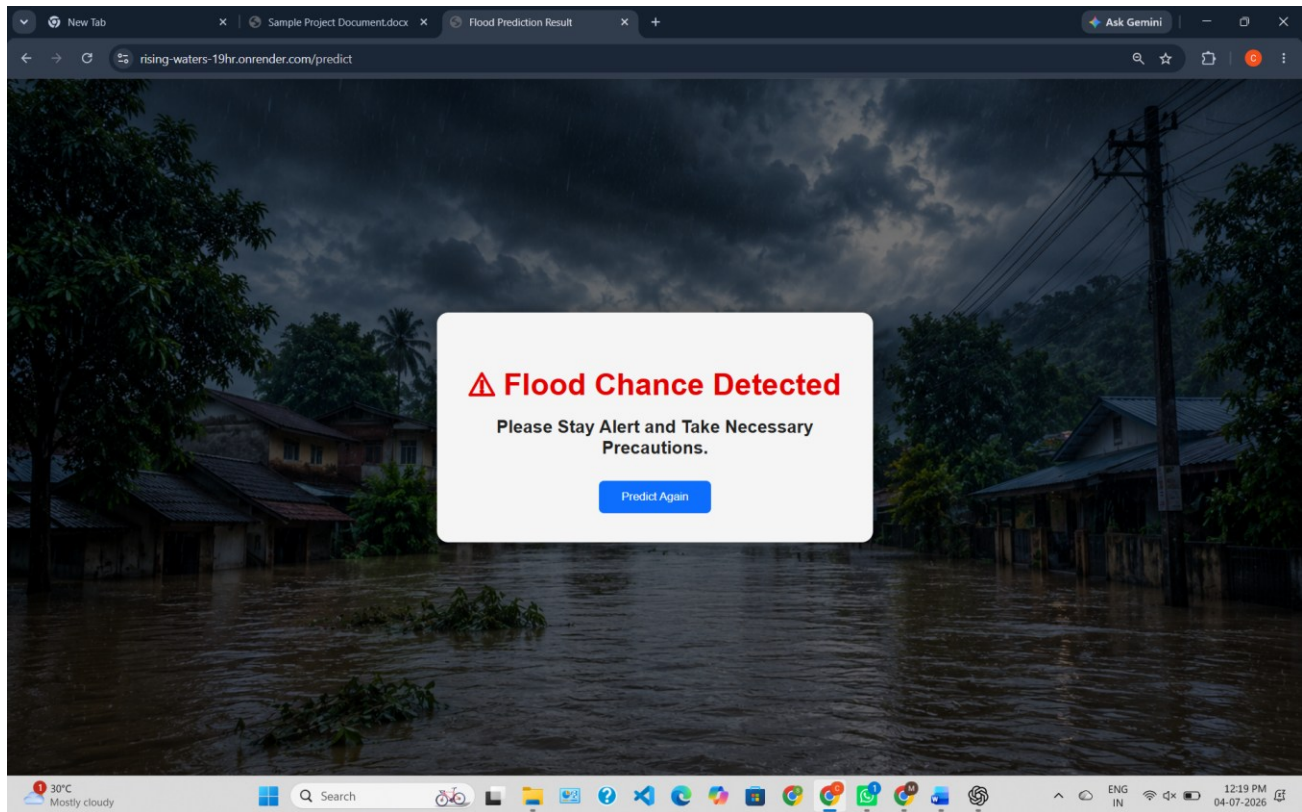
```
if prediction[0] == 1:
    result = "Flood Likely"
    result_class = "danger"

else:
    result = "No Flood Likely"
    result_class = "safe"
```

3.11 Displaying Prediction Result

Instead of redirecting users to another page, the prediction result is displayed on the same prediction page using a dynamic result card. The card changes its appearance based on the prediction outcome, using green for safe conditions and red for flood conditions.

This approach provides a smoother and more interactive user experience.



3.12 User Interface Design

The user interface was designed to provide a clean and modern appearance. A background image with a semi-transparent overlay was used to improve readability. Responsive layouts, hover effects, and professional typography were incorporated to enhance the overall user experience.

3.13 Deliverables of Milestone 3

At the end of this milestone, the following tasks were successfully completed:

- Flask application initialized.
- Trained model and scaler loaded.
- Home page developed.
- Prediction page designed.
- Input validation implemented.
- Prediction route created.
- Machine Learning model integrated with Flask.
- Dynamic result card implemented.
- Responsive user interface completed.

Milestone 4 : Testing & Performance Evaluation

4.1 Introduction

Once the Flood Prediction System was successfully developed and integrated with the trained Machine Learning model, the next step was to evaluate its functionality, reliability, and overall performance. Testing was carried out to verify that each module worked as expected and that the application generated correct predictions for different combinations of weather parameters. This milestone also focused on validating user inputs, handling unexpected situations, verifying prediction accuracy, and ensuring that the web application provided a smooth user experience. Any issues identified during testing were corrected before deployment.

4.2 Functional Testing

Functional testing was performed to verify that every feature of the application worked according to the project requirements. Each module was tested independently to ensure that it produced the expected output.

The following functionalities were tested:

- Loading the home page
- Navigation between pages
- Entering weather parameters
- Form submission
- Prediction generation
- Displaying prediction results
- Predict Again button
- Responsive user interface

All the above functionalities worked successfully without any major issues.

4.3 Input Validation Testing

The application validates user input using HTML form validation. Each field is configured with appropriate minimum and maximum values along with the required attribute. This prevents users from entering unrealistic or incomplete values.

Several test cases were executed to verify that invalid inputs were rejected and only valid values were accepted for prediction.

Test Cases

Test Case	Expected Result	Status
Empty input fields	Validation message displayed	Pass
Value below minimum limit	Input rejected	Pass
Value above maximum limit	Input rejected	Pass
Valid input values	Prediction generated	Pass

```
<label> 🌡 Temperature (°C)</label>

<input type="number"
  step="0.1"
  min="0"
  max="60"
  name="temp"
  placeholder="Example: 30"
  required>

</div>

<div class="input-box">

  <label> 💧 Humidity (%)</label>

  <input type="number"
    step="0.1"
    min="0"
    max="100"
    name="humidity"
    placeholder="Example: 75"
    required>
```

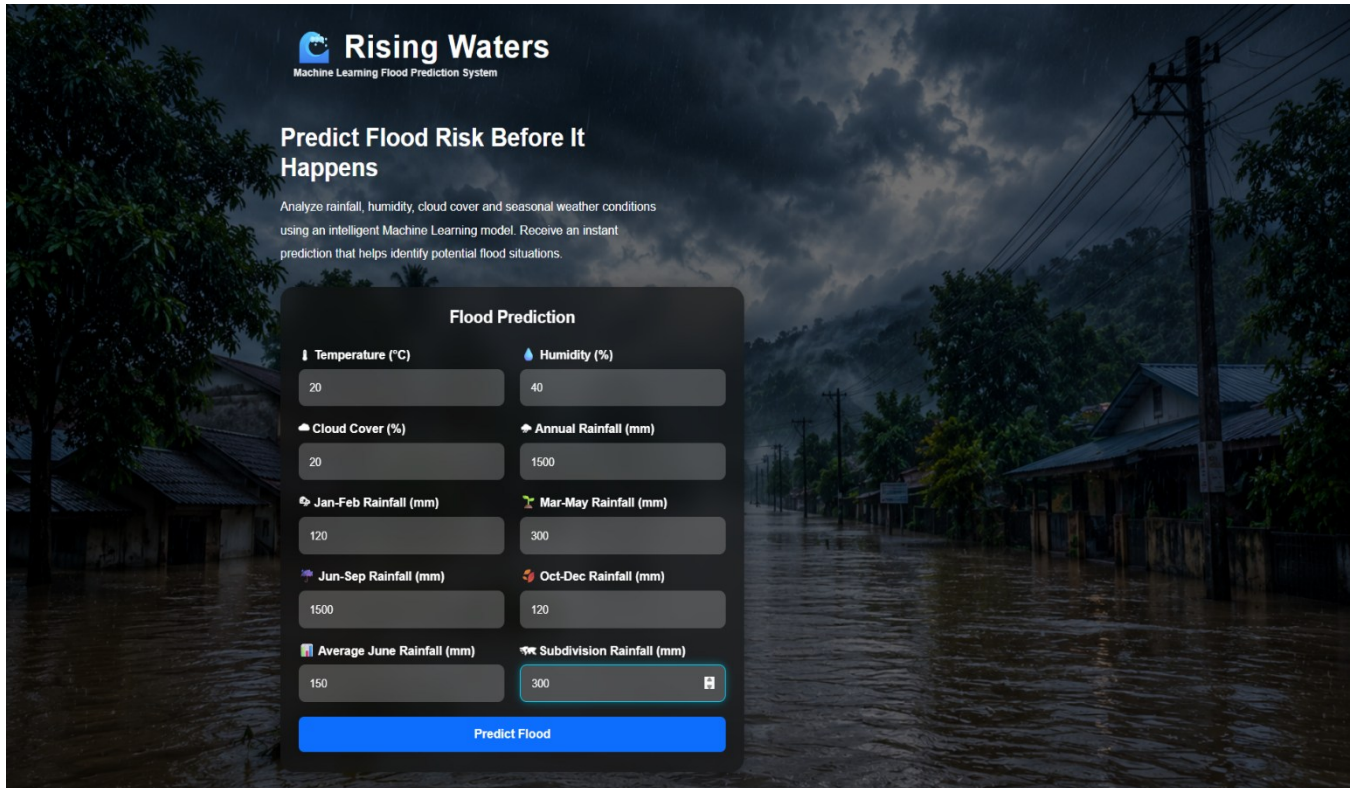
4.4 Prediction Testing

Prediction testing was carried out by entering different combinations of weather and rainfall values into the application. The Machine Learning model successfully generated predictions for all valid inputs.

Two different prediction scenarios were tested:

- Flood Likely
- No Flood Likely

The application displayed the prediction result immediately on the same page without requiring any page redirection.



Rising Waters
Machine Learning Flood Prediction System

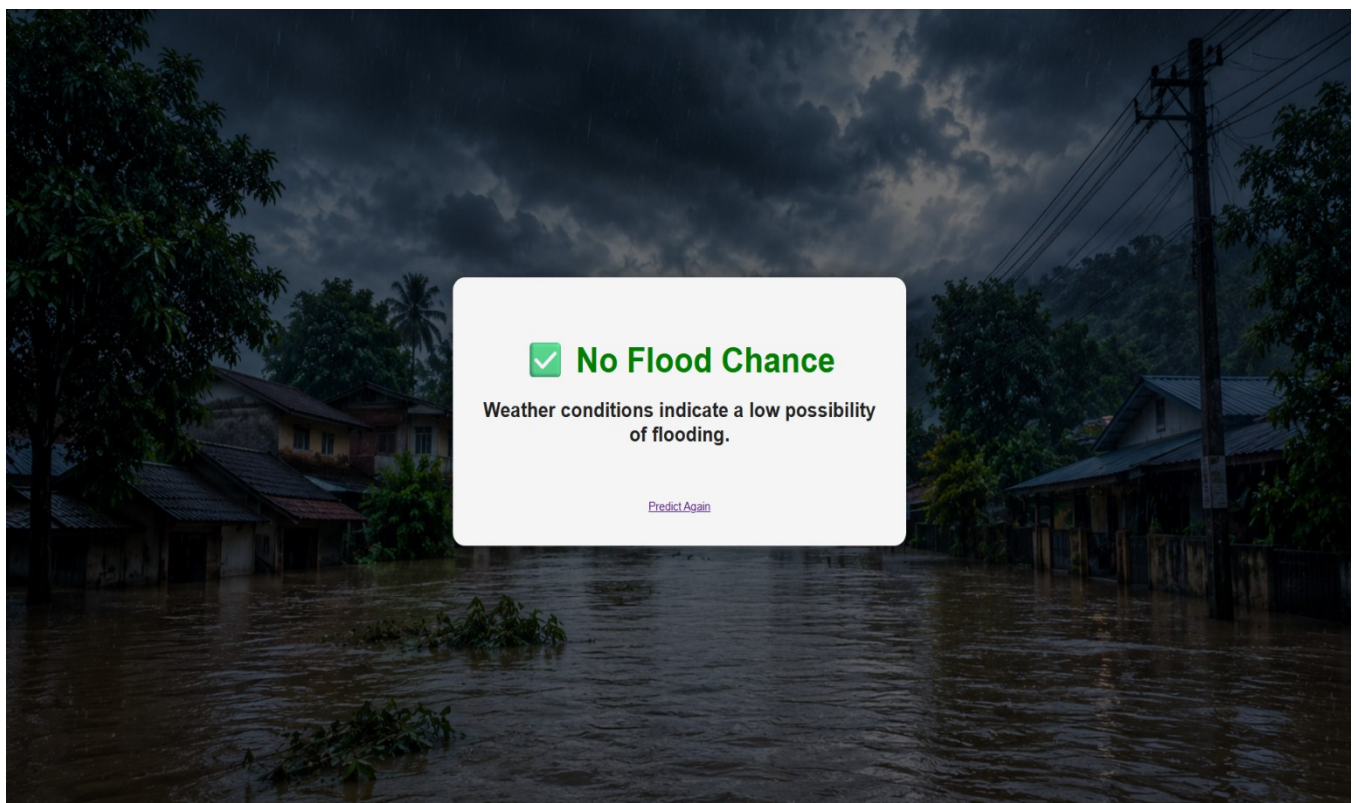
Predict Flood Risk Before It Happens

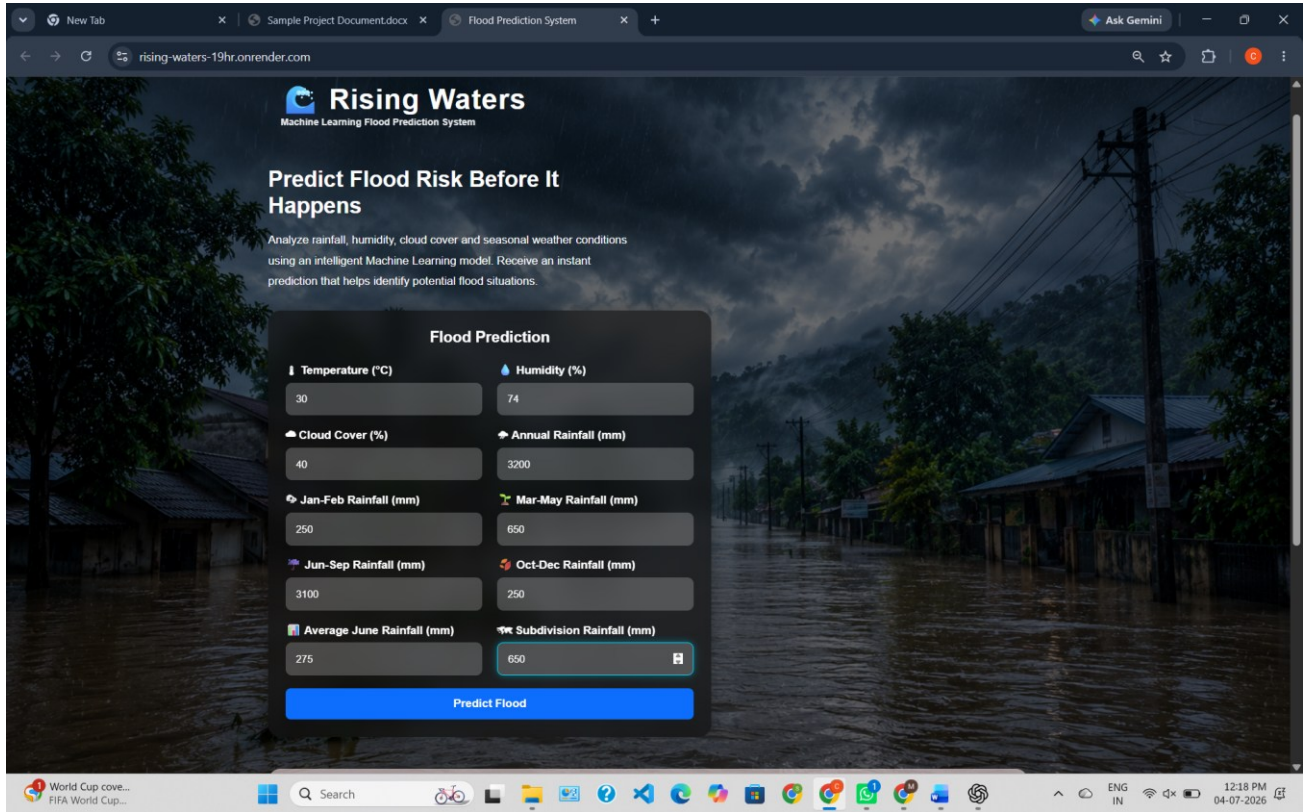
Analyze rainfall, humidity, cloud cover and seasonal weather conditions using an intelligent Machine Learning model. Receive an instant prediction that helps identify potential flood situations.

Flood Prediction

🌡️ Temperature (°C)	🌧️ Humidity (%)
20	40
☁️ Cloud Cover (%)	🌧️ Annual Rainfall (mm)
20	1500
🌧️ Jan-Feb Rainfall (mm)	🌧️ Mar-May Rainfall (mm)
120	300
🌧️ Jun-Sep Rainfall (mm)	🌧️ Oct-Dec Rainfall (mm)
1500	120
🌧️ Average June Rainfall (mm)	🌧️ Subdivision Rainfall (mm)
150	300

[Predict Flood](#)





Rising Waters
Machine Learning Flood Prediction System

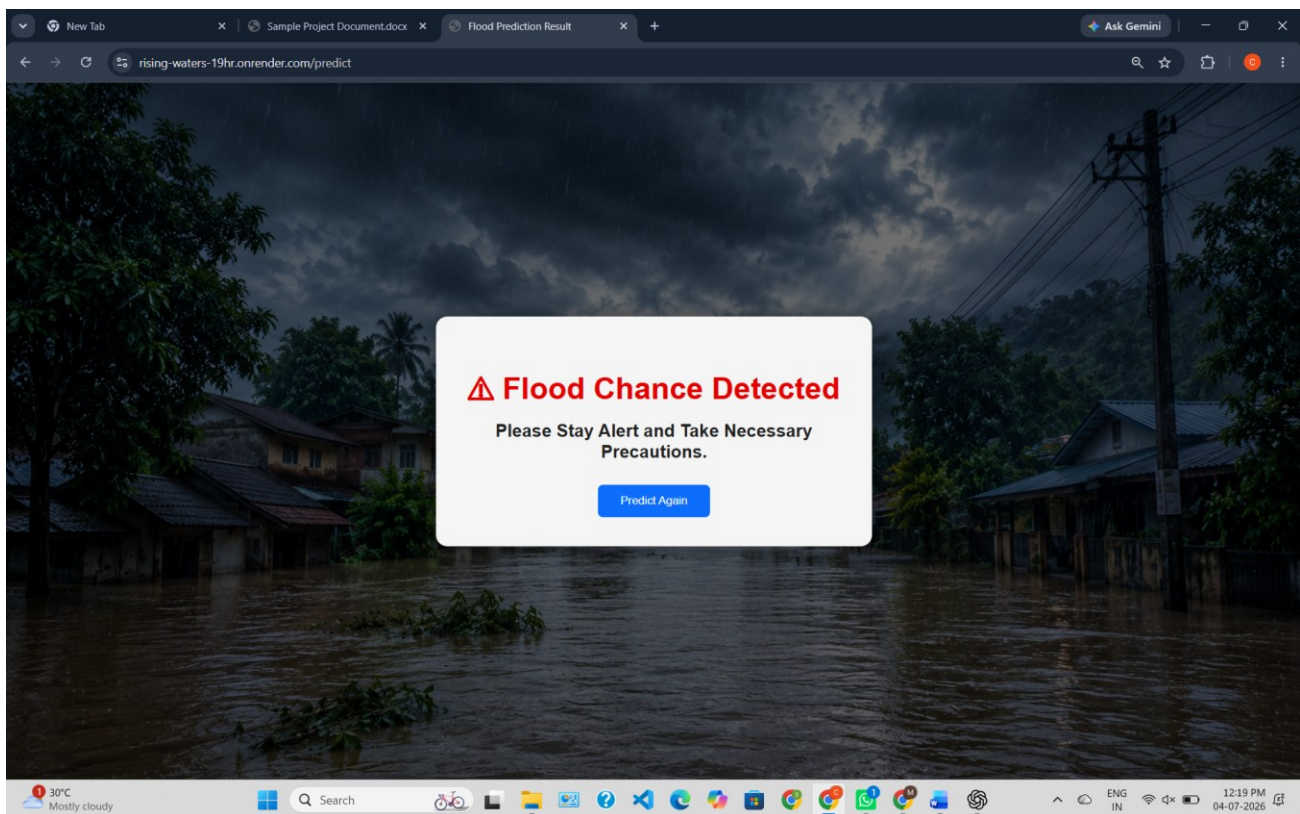
Predict Flood Risk Before It Happens

Analyze rainfall, humidity, cloud cover and seasonal weather conditions using an intelligent Machine Learning model. Receive an instant prediction that helps identify potential flood situations.

Flood Prediction

Temperature (°C)	Humidity (%)
30	74
Cloud Cover (%)	Annual Rainfall (mm)
40	3200
Jan-Feb Rainfall (mm)	Mar-May Rainfall (mm)
250	650
Jun-Sep Rainfall (mm)	Oct-Dec Rainfall (mm)
3100	250
Average June Rainfall (mm)	Subdivision Rainfall (mm)
275	650

Predict Flood



4.5 Performance Evaluation

The application's performance was evaluated under normal usage conditions. Since the trained XGBoost model is loaded only once during application startup, prediction requests are processed quickly.

The response time was observed to be within an acceptable range for an academic web application. Resource utilization remained low, and no significant delays were observed during prediction.

Performance Summary

Metric	Observed Result
Average Response Time	Less than 1 second
Prediction Accuracy	Based on trained XGBoost model
Application Stability	Stable
Error Rate	No runtime errors observed
User Experience	Smooth and responsive

4.6 Error Handling

Basic error handling was implemented to improve the robustness of the application. The prediction logic was enclosed within a try-except block to handle unexpected runtime exceptions gracefully.

If an error occurs during prediction, the application displays an informative error message instead of terminating unexpectedly.

4.7 User Interface Testing

The graphical user interface was tested to ensure consistency across different sections of the application.

The following aspects were verified:

- Navigation bar
- Background images
- Responsive layout
- Form alignment
- Button hover effects
- Result card styling
- Typography

The interface displayed correctly and provided a consistent user experience.

4.8 Bug Fixes

During development and testing, several issues were identified and resolved.

Issues Resolved

- Flask route configuration corrected.
- Prediction displayed on the same page instead of a separate result page.
- Input validation limits added.
- CSS alignment improved.
- Background image overlay adjusted.
- Responsive layout improved.
- Result card styling enhanced.

Resolving these issues improved both the functionality and usability of the application.

4.9 Testing Summary

The application was successfully tested using multiple valid and invalid input combinations. All major functionalities worked as expected, and the prediction results were generated correctly. The user interface remained responsive throughout testing, and no critical runtime errors were encountered. Overall, the application satisfied the functional and usability requirements defined during the project planning phase.

4.10 Deliverables

The following tasks were completed during this milestone:

- Functional testing completed.
- Input validation verified.
- Prediction testing performed.
- Performance evaluated.
- Error handling implemented.
- User interface tested.
- Identified bugs corrected.
- Final application validated.

Milestone 5 : Deployment & Documentation

5.1 Introduction

The final milestone focused on deploying the Flood Prediction System to a cloud platform and preparing the project for submission. After successfully developing and testing the application, the next objective was to make it accessible through the internet so that users could use the application without installing it locally.

The Flask application was deployed using Render, while GitHub was used for version control and project management. A comprehensive README file and project documentation were also prepared to help users understand, install, and execute the project.

5.2 Preparing the Project for Deployment

Before deployment, the project structure and dependencies were reviewed to ensure compatibility with the deployment platform.

The following files were prepared:

- app.py
- requirements.txt
- Procfile
- README.md
- .gitignore

These files are essential for deploying a Flask application on Render.

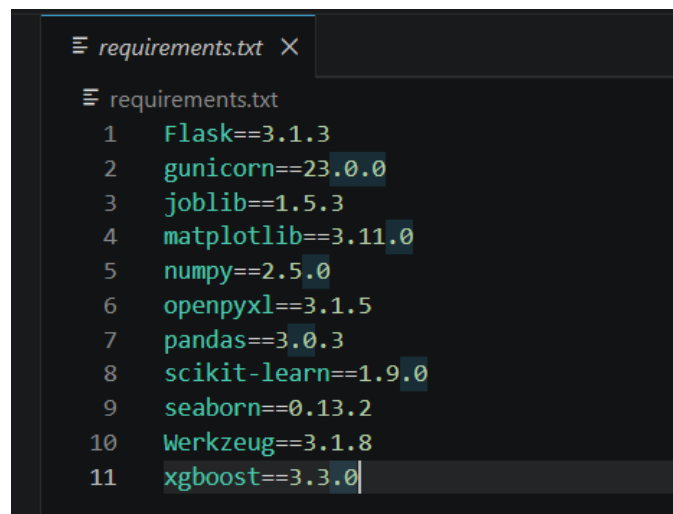
5.3 Creating the Requirements File

A requirements.txt file was created containing all Python libraries used in the project.

This file enables Render to install all required dependencies automatically during deployment.

Some of the important libraries included are:

- Flask
- XGBoost
- Scikit-learn
- NumPy
- Pandas
- Joblib
- Matplotlib
- Seaborn



```
requirements.txt
1 Flask==3.1.3
2 gunicorn==23.0.0
3 joblib==1.5.3
4 matplotlib==3.11.0
5 numpy==2.5.0
6 openpyxl==3.1.5
7 pandas==3.0.3
8 scikit-learn==1.9.0
9 seaborn==0.13.2
10 Werkzeug==3.1.8
11 xgboost==3.3.0
```

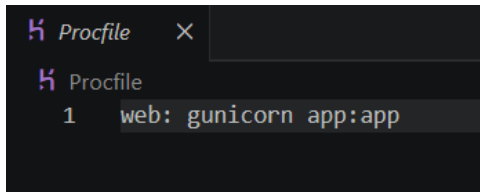

5.4 Creating the Procfile

A Procfile was added to specify the command required to start the Flask application on Render. The Procfile instructs the deployment platform to launch the application using Gunicorn.

Example:

web: gunicorn app:app

This configuration ensures that the application starts correctly in the cloud environment.



```
Procfile
1 web: gunicorn app:app
```

5.5 Version Control Using GitHub

GitHub was used to maintain the complete source code and version history of the project.

After verifying the project locally, all files were committed and pushed to the GitHub repository using Git.

The repository contains the complete source code, trained models, documentation, and project resources.

Version control made it easier to manage updates and collaborate if required.

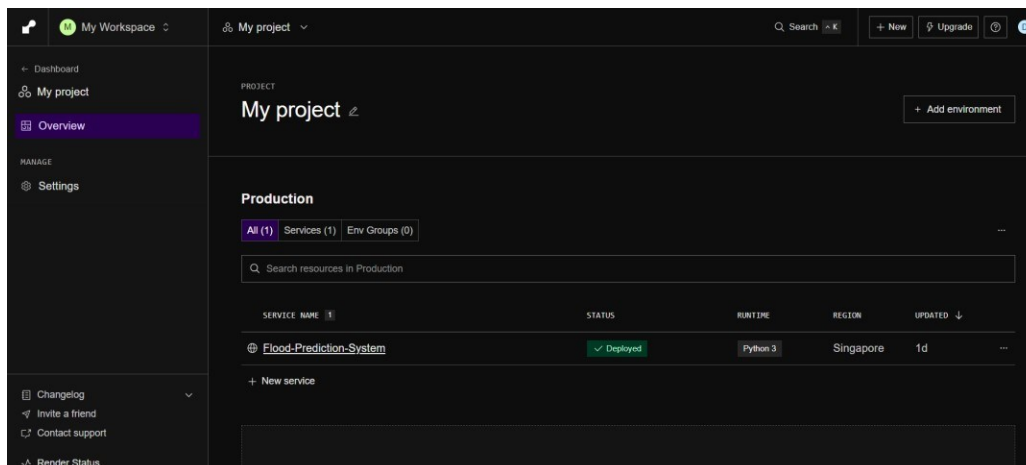
5.6 Deploying the Application on Render

The GitHub repository was connected to Render to deploy the application.

Render automatically installed the project dependencies using the requirements.txt file and launched the Flask application using the Procfile.

Once deployment was completed successfully, a public URL was generated.

This allows users to access the application directly from a web browser.



5.7 Live Application

After deployment, the application was tested through the Render URL.

The deployed application provides the same functionality as the local version.

Users can:

- Open the home page
- Navigate to the prediction page
- Enter weather parameters
- Generate predictions
- View results instantly

The deployed application can be accessed using the following URL:

<https://rising-waters-19hr.onrender.com/?>

5.9 Final Verification

After deployment, the application was tested once again to verify that all functionalities worked correctly.

The following checks were performed:

- Home page loaded successfully.
- Prediction page opened correctly.
- Form accepted valid input values.
- Prediction generated successfully.
- Result displayed correctly.
- No runtime errors occurred.

The application performed as expected in the deployed environment.

5.10 Deliverables

At the completion of this milestone, the following deliverables were successfully prepared:

- Flask application deployed on Render.
- GitHub repository completed.
- README documentation finalized.
- Project dependencies documented.
- Profile configured.
- Live application tested.
- Complete project report prepared.
- Final project submitted.

Overall Project Conclusion

The **Flood Prediction System** was successfully designed, developed, tested, and deployed as a Machine Learning-based web application. Historical weather and rainfall data were analyzed to train multiple classification models, and **XGBoost** was selected as the final model due to its superior performance. The trained model was integrated with a Flask backend and presented through a responsive HTML and CSS interface.

The deployed application allows users to enter weather parameters and receive real-time flood predictions through an intuitive web interface. The project demonstrates the complete machine learning lifecycle, including data preprocessing, model development, web integration, testing, deployment, and documentation.